Path Reconstruction in Dynamic Wireless Sensor Networks Using Compressive Sensing

Zhidan Liu*† Zhenjiang Li† Mo Li† Wei Xing* Dongming Lu*
*College of Computer Science and Technology, Zhejiang University, China
†School of Computer Engineering, Nanyang Technological University, Singapore
danielliu@zju.edu.cn, {lzjiang, limo}@ntu.edu.sg, {wxing, ldm}@zju.edu.cn

ABSTRACT

This paper presents CSPR, a compressive sensing based approach for path reconstruction in wireless sensor networks. By viewing the whole network as a path representation space, an arbitrary routing path can be represented by a path vector in the space. As path length is usually much smaller than the network size, such path vectors are sparse, i.e., the majority of elements are zeros. By encoding sparse path representation into packets, the path vector (and thus the represented path) can be recovered from a small amount of packets using compressive sensing technique. CSPR formalizes the sparse path representation and enables accurate and efficient per-packet path reconstruction. CSPR is invulnerable to network dynamics and lossy links due to its distinct design. A set of optimization techniques are further proposed to improve the design. We evaluate CSPR in both testbed-based experiments and large-scale trace-driven simulations. Evaluation results show that CSPR achieves high path recovery accuracy (i.e., 100% and 96% in experiments and simulations, respectively), and outperforms the state-of-the-art approaches in various network settings.

Categories and Subject Descriptors

C.2.1 [**Network Architecture and Design**]: Wireless Communication; C.2.3 [**Network Operations**]: Network monitoring

Keywords

Packet path reconstruction; wireless sensor networks; compressive sensing; bloom filter

1. INTRODUCTION

The per-packet routing path serves as the meta-information for understanding detailed Wireless Sensor Networks (WSNs) behaviors in many network maintenance and diagnosis situations, e.g., routing dynamics [33], detections on wormholes [9] or packet loss holes [32], end-to-end packet transmission delay [29] or even per-hop per-packet transmission delay [13], network diagnosis [18] [26], etc. Reconstructing per-packet routing path information, however, has been known non-trivial. WSNs are self-organized and usually deployed in dynamic environments. The underlying network topology constantly changes and no fixed routing path can be expected for each node [30]. A straightforward solution to reveal the packet path is to record the complete path during packet forwarding, e.g., storing the ID sequence of all relay nodes, in each packet. The introduced overhead linearly grows with the routing path length, far from scalable.

There have been many efforts made to address the per-packet path reconstruction problem in WSNs. The method that identifies packet paths via hash values triggers disastrous computation overhead [19]. Some methods reconstruct path information by leveraging inter-packet correlation in sufficiently stable and reliable networks [14] [16]. However, according to our investigation on the practical packet trace from CitySee [22], a real-deployed and large-scale WSN, we observe non-negligible topology variation (e.g., up to 28% packets experienced parent changes) and high packet loss (e.g., up to 55% packets lost for some nodes) all the time. Both topology instability and packet loss significantly deteriorate existing path reconstruction methods in practical WSNs. To cope with above issues, we attack the path reconstruction problem from a new perspective, which requires no inter-packet correlations and thus makes the solution insensitive to network dynamics and lossy links.

The key insight of our design is as follows. The length of a routing path is usually much smaller than the network size. As a concrete example, the maximum path length reported in CitySee [22] is only 20 hops in comparison with its network size of 1200 nodes. Therefore, we can construct a *path representation space*, the number of whose dimensions equals to the total number of nodes in the network. In such a representation space, an arbitrary routing path can be represented by a *path vector*, where each element corresponds to a node in the network. The path vector sets the hop numbers for nodes on the path and zeros for those not involved in the path. As the path length is much smaller than the network size, such path vectors are thus **sparse**, i.e., the majority of elements are zeros. The path reconstruction becomes a problem of unveiling all existing path vectors hidden in the representation space. If all non-zero elements of a path vector can be encoded (with few bytes) into the packets forwarded along the path, we can recover the path vector (and thus the represented routing path) based on a small amount of packets using compressive sensing technique [5] [12].

In this paper, we propose a **C**ompressive **S**ensing based **P**ath **R**econstruction method, CSPR, which formalizes the sparse path representation and leverages compressive sensing to recover per-packet routing path. CSPR lets intermediate nodes briefly annotate the transmitted packets and classifies packets traveling along different paths into different groups. For a particular path, the forwarded packets encode independent observations and CSPR performs compressive sensing to recover the path when a certain amount of pack-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
MobiHoc'14, August 11–14, 2014, Philadelphia, PA, USA.
Copyright 2014 ACM 978-1-4503-2620-9/14/08 ...$15.00.
http://dx.doi.org/10.1145/2632951.2632967 .

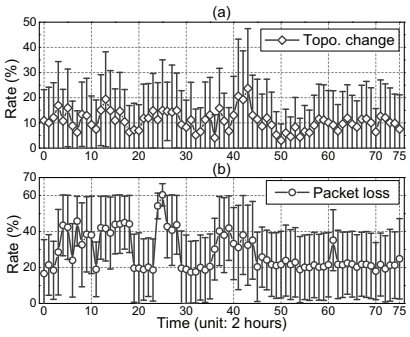


Figure 1: Average (a) topology change rate and (b) packet loss rate of all nodes at each time window.

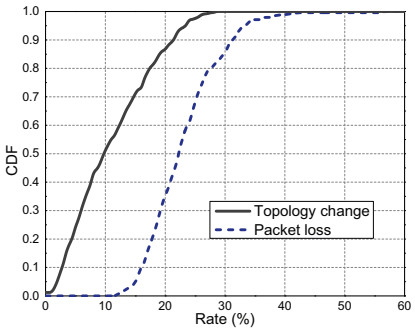


Figure 2: CDF of topology change rates and packet loss rates of all nodes in the CitySee packet trace.

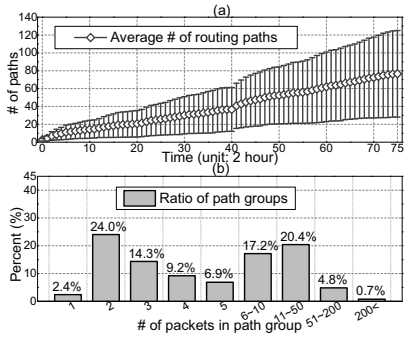


Figure 3: (a) Average number of paths for each node. (b) The distribution of packet volumes for all path groups.

ets (and the annotations) are collected at the sink. The path reconstruction by CSPR requires no inter-packet correlations and utilizes only a small number of received packets. CSPR is thus invulnerable to topology dynamics and lossy links. On the protocol level, CSPR introduces only small and fixed overhead in annotating each packet, which could be optimized accordingly for practical WSNs (e.g., 8 bytes per packet for a network with 245 nodes). In addition to the basic design, we further propose a set of optimization techniques to gradually shrink the representation space and heuristically scan possible paths for all unrecovered path vectors through the network topology learnt from already reconstructed routing paths. The numbers of packets needed for remaining path reconstructions are lowered and processing is thus accelerated. To examine the performance of CSPR, we first evaluate our method using a 29 TelosB mote testbed. The experiment results validate the feasibility and applicability of CSPR in practice. We further conduct extensive and large-scale trace-driven simulations to examine the efficiency and salability of CSPR. Compared to the state-of-the-art methods, CSPR achieves higher path recovery accuracy (i.e., 100% and 96% for experiments and simulations, respectively) with comparable overhead (8 extra bytes per packet).

The rest of this paper is organized as follows. The path reconstruction problem and the motivation of our design are presented in Section 2. The design of CSPR is detailed in Section 3. In Section 4, we evaluate CSPR through testbed experiments and trace-driven simulations. We review the related works in Section 5. Section 6 concludes this paper.

2. PROBLEM STATEMENT AND MOTIVATION

2.1 The path reconstruction problem

In a WSN, all sensor nodes generate and relay packets to the sink along some routing paths [28]. At the sink, a path reconstruction method is desired to recover the routing path each packet traveled. One packet path is an ID sequence from the source of the packet to the sink, including IDs of all intermediate nodes relaying this packet and their hop numbers as well.

There have been many efforts made to address the path reconstruction problem (as reviewed in Section 5). Two state-of-the-art methods, MNT [16] and Pathfinder [14], have been recently proposed. MNT [16] reconstructs per-packet path by exploiting *inter-packet correlation*, i.e., a relayed packet and its adjacent packets locally generated at any node i are usually forwarded to the same next hop. Such local packets serve as *anchors* of the relayed packet at node i. As the first-hop receiver is recorded in packets, the path of a packet can be obtained by concatenating the first-hop receivers of all its anchors. Improving on MNT, Pathfinder [14] tolerates certain inconsistence in inter-packet correlation via explicitly recording inconsistence in packets. The reconstruction failure occurs once the inconsistence exceeds the tolerance capacity. To accurately locate anchors, Pathfinder further imposes the packet generation rate of each node to be identical and fixed. Both MNT and Pathfinder require stable network topology such that inter-packet correlation can be captured. The practical WSNs, however, behave dynamically and the wireless links are far from stable [17] (as we will demonstrate in next subsection). Both network dynamic and packet loss have strong impacts on the anchor identifications, and thus deteriorate the performances of MNT and Pathfinder.

2.2 How packet routing behaves in practical WSNs

We investigate the packet trace from a real-deployed and large-scale WSN CitySee [22], and discuss how practical packet routing behaviors impact the path reconstruction performances of the state-of-the-art methods as well. The CitySee, deployed in Wuxi city, China, contains more than 1200 sensor nodes for monitoring urban environmental factors, including carbon dioxide, temperature, humidity, light, etc. Sensor nodes generate data every 10 minutes, encapsulate data into a single packet, and transmit packets to the sink with CTP [15] in a multi-hop manner. Each packet possesses a default CTP packet header including following common fields: *source address*, ID of the node generating this packet; *sequence number*, order of the packet generated from the source; *first-hop receiver*, parent node ID of the packet's source; and *hop count*, length of path traveled by the packet. Each packet further records the first 10-hop node IDs for future analysis. The packet trace from a subnetwork of 245 nodes with a collection period of one week is available for our study. Such partial network covers about 16 km^2 area with the average and maximum path length as 7 hops and 12 hops, respectively.

With CitySee packet trace, we examine the *topology change* and *packet loss*, which are most relevant to the stability of WSNs [6]. For each node, one topology change is indicated by the first-hop receiver difference between two consecutively transmitted packets. The topology change rate of a network is defined as the average of ratios $\frac{\#\ of\ pkts\ with\ topo.\ changes}{total\ \#\ of\ pkts\ received} \times 100\%$ for each node within certain time duration. Similarly, for each node, one packet loss is indicated by a sequence number missing. The packet loss rate of a network is

further defined as the average of ratios $(1-\frac{\#\ of\ pkts\ received}{\#\ of\ pkts\ expected\ to\ receive})\times 100\%$ for each node within certain time duration.

Topology change rate indicates the topology stability of a network, which reflects the validity of the inter-packet correlation assumption made in existing path reconstruction methods [14] [16]. Fig. 1 (a) depicts the topology change rate in CitySee with 2 hour time window of measurement. From the figure, we observe that the average and maximum topology change rates at all time windows are about 12% and 48%, respectively. In Fig. 2, we summarize the CDF of topology change rates of all nodes during one week. For most nodes, the rate is as high as 10% and the maximum rate reaches up to 28%. Due to the instable topology, each node would transmit packets to the sink via different paths. In Fig. 3 (a), we plot the number of routing paths formed in the packet trace as time goes by. The average number of paths starting from a node persistently increases and finally on average each node has 76 paths. All of these evidences demonstrate that network topology suffers from severe instability and the inter-packet correlation may not be well validated in practice. In Section 4, we have implemented both MNT and Pathfinder. By analyzing their detailed executions, we find that the topology dynamics could solely cause about 10% and 1% of anchor misidentifications, which directly lead to path reconstruction failures. Pathfinder outperforms MNT at the cost of explicitly recording certain topology changes in each packet.

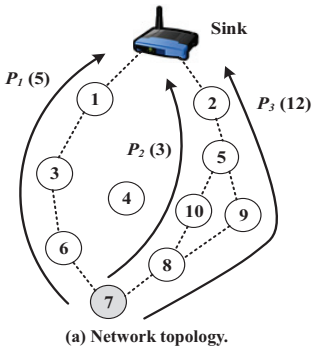
Packet loss rate reflects the reliability of packet receptions [11]. Packet losses will hide the inter-packet correlation, e.g., anchor availability, and deteriorate performances of methods relying on such correlation. Fig. 1 (b) depicts the packet loss rate in the packet trace, with 2 hour time window. From the figure, we see that the packet loss rate changes rapidly with the average ranging from 17% to 69%. In Fig. 2, we summarize the CDF of packet loss rates of all nodes during one week. The distribution mainly concentrates in the range between 15% and 35%, with the maximum rate as high as 55%. By analyzing the detailed executions of MNT and Pathfinder, we find that with such a high packet loss rate, about 49% and 35% packets fail to identify anchors, which directly result in path reconstruction failure for those packets.

With the joint impacts of topology dynamics and packet losses, we find that in the large-scale and dynamic WSN CitySee, 59% and 36% packet paths might not be successfully reconstructed by the state-of-the-art methods MNT and Pathfinder, respectively. This motivates us to explore a solution insensitive to both network dynamics and lossy links.

2.3 Path reconstruction from sparse path representation

Sparse representation of routing paths. Since sensory data in WSNs are usually collected with a direct acyclic graph (DAG) routing structure (e.g., data collection tree), the path length is thus in the order of $O(\lg(N))$, where N is the total number of nodes in the network. According to the statistical results of all paths formed in CitySee packet trace, the path length ranges between 2 hops and 12 hops, which are much smaller than the network size 245. We can construct a path representation space $\mathbb{N}$. The dimensionality of $\mathbb{N}$ equals to the total number of nodes in the network and each dimension corresponds to one node. In such a representation space, any routing path can be presented by a path vector. According to whether a node is involved in the routing path, the path vector sets either hop number or zero for its corresponding element. As the path length is much smaller than the network size, the path vector is thus sparse, i.e., the majority of elements in the vector are zeros.

Fig. 4 illustrates the sparse path representation with a network containing 10 nodes. We construct the path representation space



(a) Network topology.

$\mathbb{N}$	n_1	n_2	n_3	n_4	n_5	n_6	n_7	n_8	n_9	n_{10}
$\mathbf{s_1}$	3	0	2	0	0	1	0	0	0	0
$\mathbf{s_2}$	0	4	0	0	3	0	0	1	0	2
$\mathbf{s_3}$	0	4	0	0	3	0	0	1	2	0

(b) Sparse representation of routing paths.

Figure 4: (a) A simple sensor network topology. Source node 7 transmits 20 packets to the sink via 3 different routing paths, i.e., 5 packets on path P_1, 3 packet on path P_2 and 12 packets on path P_3. (b) The sparse representations of the 3 routing paths in the whole network space.

$\mathbb{N} = [n_1, n_2, n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}]^T$ including all nodes. An arbitrary in-network path can be represented by a path vector in $\mathbb{N}$. For example, path P_1 is represented as $\mathbf{s_1} = [3,0,2,0,0,1,0,0,0,0]^T$. Node 7 is the source node and could be known from the packet header. Hence the element n_7 is not considered in $\mathbf{s_1}$ and thus assigned as 0. As nodes 6 , 3 and 1 serve as the first, second and third hop relay, respectively, in path P_1, thus $n_6 = 1$, $n_3 = 2$ and $n_1 = 3$. All other elements irrelevant to P_1 are zeros in $\mathbf{s_1}$. The sparsity of this path vector is 3 (out of 10). Other path vectors, e.g., $\mathbf{s_2}$ and $\mathbf{s_3}$, can be similarly constructed.

Compressive sensing based path reconstruction. Based on sparse path representation, the path reconstruction thus becomes a problem of unveiling all existing path vectors hidden in the path representation space $\mathbb{N}$. Packets from the same source may travel different paths to the sink, while the paths implicitly classifies packets into different *path groups*, i.e., all packets in one group travel exactly the same path. After the paths of all path groups get reconstructed, the path of each packet is obtained as well.

For the path vector of a routing path, if its non-zero elements can be encoded into each packet forwarded along the path, it is viable to recover the vector (and thus the represented path) based on a small amount of packets using compressive sensing [5] [12]. In particular, for any path vector $\mathbf{s}$ in space $\mathbb{N}$ with sparsity k, where $k \ll N = |\mathbb{N}|$, the compressive sensing theory states that M, instead of N, independent equations are sufficient to solve the N unknowns in $\mathbf{s}$, where $M \ll N$. The M independent equations can be acquired by projecting $\mathbf{s}$ to a measurement matrix Φ: $Y = \Phi\mathbf{s}$, where Y is an M-dimension vector and Φ is an $M \times N$ matrix. If Φ satisfies the *Restricted Isometry Property* [4], $\mathbf{s}$ can be exactly recovered by solving following l_1-minimization problem:

$$\hat{\mathbf{s}} = \arg\min_{\mathbf{s}\in R^N} \|\mathbf{s}\|_{l_1} \quad s.t.\ Y = \Phi\mathbf{s},$$

when $M \geq ck\log(\frac{N}{k})$, where c is a small positive constant [4]. A variety of algorithms have been proposed for solving above optimization problem.

The above compressive sensing based approach makes path reconstruction for each path group independent, and can recover the path for a group of packets once sufficient packet are accumulated. As a result, this approach requires no inter-packet correlation, which makes itself inherently invulnerable to network dynamics and lossy links. On the other hand, once a path is recovered, the routing path for all future packets residing in the same path group

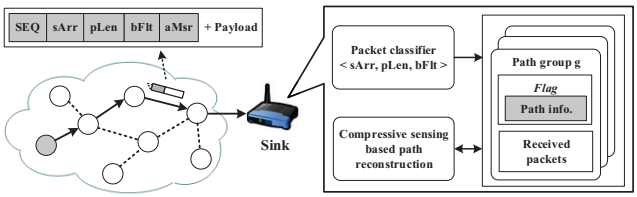


Figure 5: The system architecture of CSPR.

becomes immediately available, which avoids repeatedly triggering path reconstruction procedure for each received packet and largely reduces the computation overhead.

Design challenges. Translating the idea of compressive sensing based per-packet path reconstruction to a practical system, however, encounters a set of challenging issues as follows.

1) *Accurate packet classification*. Packet classification into each path group must be accurate. Fig. 3 (a) plots the average number of paths (each path one path group) formed in the packet trace as more and more packets are received by the sink. According to the statistics, we observe significant differences existing in the number of paths for each node, e.g., the maximum number of paths of some node could even reach 199. It is vital to distinguish each individual path and classify the packets into the right group. Packet misclassification will lead to path reconstruction error for one group. Therefore, packet classification must have high accuracy.

2) *Lightweight per-packet annotation*. To enable the compressive sensing based path reconstruction, packets need carry encoded information for all non-zero elements of the path vectors representing their traveling paths. In particular, when one packet is relayed by an intermediate node, the node needs to annotate its hop number along the path into the packet header. As a result, packet header is updated hop by hop. The annotation overhead must be small. In particular, the annotation field in the packet header should be small in size and maintain constant (not increase with the path length). In addition, updating should be performed in a distributed manner without introducing any centralized control.

3) *Short per-packet path recovery delay*. From Fig. 3 (b), we observe that the packet volumes for all path groups within one week are highly heterogenous. About 50% path groups include fewer than 5 packets while some path groups (about 5.5%) contain more than 50 packets. To perform compressive sensing based path reconstruction for one path group, the number of packets (one annotation as one measurement for each packet) accumulated in this group should be at least M to ensure a good recovery quality. Some path groups, however, cannot accumulate sufficient packets even after a long time. As practical requirement, per-packet path recovery delay should not be excessively long.

3. DESIGN OF CSPR

CSPR consists of two parts, the in-network part for path information encoding and the server part for per-packet path reconstruction. The system architecture of CSPR is depicted in Fig. 5. We will present the system overview first and then detail each component in following subsections.

3.1 CSPR overview

Several fields in the packet header are used by CSPR to carry packet information, as depicted by Fig. 5. SEQ is the packet sequence number. sArr is source address of the packet. pLen records the path length. bFlt is a bloom filter to space-efficiently record the IDs and corresponding hop count information of all relay nodes. aMsr stores the encoded measurement along the path. All the five fields are initialized at the source node. In particular, SEQ and sArr keep unchanged after initialization, whereas pLen, bFlt and aMsr are updated at each intermediate hop. Note that only two fields, bFlt and aMsr, are additionally introduced by CSPR. SEQ, sArr and pLen can be usually found in the default packet header, e.g., CTP packet header. The extra overhead to each packet is thus slight, e.g., 6 bytes of bFlt and 2 bytes of aMsr for a network with 245 nodes. We detail the in-network updating of packet header in Section 3.2.

CSPR adopts a 3-tuple key, $<$ sArr, pLen, bFlt $>$, to identify the path of a packet. For all received packets, CSPR first distinguishes their paths according to sArr, and then differentiates those from the same source based on pLen. Finally, bFlt is used to distinguish paths owning the same sArr and pLen. If two packets have the same 3-tuple key, they are considered to travel the same path and will be classified into the same path group. At the sink, CSPR maintains a database, where each entry is indexed via the 3-tuple key and corresponds to a unique path group. When a packet is received, CSPR extracts the 3-tuple key from packet header and looks for a matched entry in the database. If the matched entry has already recovered the path, the path for the packet becomes immediately available. If an entry is matched yet the path is not ready, CSPR launches path reconstruction when sufficient packets are accumulated. If no entry matched, CSPR creates an entry for the new path group indexed by the 3-tuple key of the packet. We detail the compressive sensing based path reconstruction component in Section 3.3.

As improvements on the basic design, a set of optimization techniques is proposed to gradually shrink the path representation space and reduce the sparsity of unrecovered path vectors. The number of packets needed by compressive sensing is accordingly lowered such that the remaining path reconstructions are accelerated. In addition, CSPR can launch a remedy scheme if some path groups fail to recover their paths after an excessive long delay. We detail those components in Section 3.4.

3.2 In-network path information encoding

In this subsection, we introduce the in-network updating of the last three fields pLen, bFlt and aMsr in turn.

Updating of pLen. The pLen field of each packet is initialized to 0 by the source and increased by one at each intermediate hop along the path. At each intermediate hop, pLen is updated prior to both bFlt and aMsr as the updating of latter two fields relies on the new pLen value. When a packet arrives at the sink, we can know the packet path length through the bFlt field, while we can not infer the hop count information for each intermediate node along the path relying on this filed at the server side.

Updating of bFlt. Bloom filter is an L-bit array associated with H independent hash functions, where L and H are two parameters to be determined. The bFlt field of each packet accommodates an L-bit array, and sensor nodes use the same set of H independent hash functions $f_i(\cdot), i = 1,2,\cdots,H$, to update bFlt. Different arrays represent different path information. Initially, all L bits in the bFlt field of a packet header are set to 0. At each intermediate hop, the node compresses its existence into the bFlt field as follows. H hash values $v_i = f_i(d \times h) \in \{0,1,\cdots,L-1\}, i = 1,\cdots,H$, are first obtained by feeding the product of node ID d and hop count h in the pLen field to the H hash functions. Then the v_i bits of the bFlt field are set to 1 by that relay node, $i = 1,2,\cdots,H$.

Updating of aMsr. The aMsr field in each packet is also initialized as 0 by the source and updated along the path. At each intermediate hop, the node encodes its hop number along the path in aMsr. In particular, the node multiplies the updated pLen value

with a random coefficient and adds the product with current aMsr value. We design such field for the purpose of recovering sparse path vector via compressive sensing technique.

When CSPR later recovers the path for a set of received packets in one path group, it solves equation $Y = \Phi\mathbf{s}$ to obtain $\mathbf{s}$ using compressive sensing technique. The path vector $\mathbf{s}$ is a column vector with N elements. Each element represents one node in the path representation space $\mathbb{N}$. If a node is included in the path represented by $\mathbf{s}$, the corresponding element indicates its hop number; Otherwise, element is zero. In the equation, each element in Y is the final aMsr value of one packet and the corresponding row in Φ can be represented by $\phi = [\alpha_1, \alpha_2, \cdots, \alpha_N]$, where α_j means that if node j relays the packet, α_j will be the coefficient multiplied with pLen value, $j = 1, 2, \cdots, N$. The product of Φ and $\mathbf{s}$ essentially replays the updating process of aMsr along the path.

Before the reconstruction of $\mathbf{s}$, we are not aware which α_j finally participates in the aMsr updating. We have to provide complete α_j in ϕ for the compressive sensing recovery. To avoid explicitly acquiring such information from each node in the network, we introduce a dictionary based strategy working in a fully distributed manner. Each node i is configured with a coefficient dictionary $\mathcal{D}_i$, stored in the RAM. To update the aMsr field for a packet, node i multiplies the updated pLen value with the n-th coefficient in $\mathcal{D}_i$, where $n = (\mathsf{SEQ}\ mod\ |\mathcal{D}_i|)$ and $|\mathcal{D}_i|$ is the size of $\mathcal{D}_i$. Each $\mathcal{D}_i$ is made up by Gaussian random numbers and the CSPR server is aware of the dictionaries of all nodes in the network. Coefficients in all dictionaries are generated independently. As a result, different nodes have different dictionary elements. In our current design, each coefficient occupies 2 bytes and each dictionary contains 100 independent coefficients. The 200-byte storage overhead accounts for only 2% storage occupancy to commercial sensor motes, e.g., TelosB with 10 KB RAM [1]. We could synchronize the generation of random numbers between nodes and server via global seed [20], while at the cost of reducing coefficient randomness.

Fig. 6 illustrates the path information encoding for a packet p on path P_3 in Fig. 4. For ease of illustration, we simply set $L = 8$ and $H = 2$ for the bloom filter. Source node 7 generates packet p with SEQ equalling to 3 and initializes sArr, pLen, bFlt, and aMsr to 7, 0, 0x00, and 0, respectively. At each intermediate hop, pLen, aMsr and bFlt are updated. For instance, at node 5, pLen is increased from 2 to 3. Two hash values are $v_1 = 1$ and $v_2 = 4$ ($v_1, v_2 \in \{0, 1, \cdots, 7\}$). The bFlt is thus updated by setting the first and fourth bits to 1. The aMsr is updated by adding the product of the updated pLen (i.e., 3) and the third coefficient in $\mathcal{D}_5$ to current aMsr value.

3.3 Compressive sensing based path reconstruction

In this subsection, we first present the packet classification mechanism in CSPR, and then detail the compressive sensing based path reconstruction with path verification scheme to ensure the reconstruction correctness.

Packet classification. For each received packet, CSPR extracts the 3-tuple key, $<$ sArr, pLen, bFlt $>$, from the packet header and then classifies it into a path group. A path group is designed to contain packets traveling the same path. At the sink, CSPR manages all path groups with a database. One database entry is indexed via the 3-tuple key and corresponds to a unique path group. Each entry is further allocated with a piece of buffer to accommodate the packets belonging to this group. An entry also has an indicator *Flag* to tell whether the path gets recovered. When a packet is received by sink, CSPR extracts the key from the packet header and looks for a matched entry. If a matched entry exists, the packet is inserted into

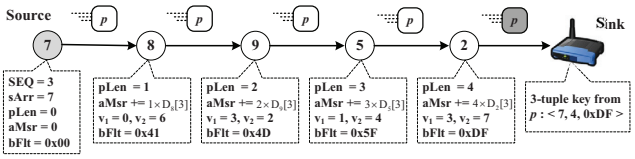


Figure 6: The path information encoding process for a packet p on path P_3 in Fig. 4. The SEQ, sArr, pLen, aMsr and bFlt fields are all initialized at source node 7, and the last three fields are updated hop-by-hop as depicted in the attached box at each intermediate hop. CSPR can extract a 3-tuple key from packet p at the sink.

the allocated buffer. Furthermore, in case the indicator *Flag* is *true*, CSPR checks whether the recovered path of this group is valid for the packet through the path verification component (as described later). When the *Flag* is *false*, however, CSPR will recover the path if the amount of packets accumulated in the group is sufficient (the number will be given later). On the other hand, if no entry matched, CSPR creates an entry for this new path group, initializes *Flag* as *false*, and inserts the packet into the buffer.

The packet classification using the 3-tuple key might lead to misclassification since the comparison result between two bloom filers could be false negative, i.e., different paths possess the same L-bit array. However, with a proper parameters L and H setting of the bloom filter, misclassification rate could be low. The setting of L trades off between packet overhead and classification accuracy. A lager L leads to a lower false negative probability yet more overhead to each packet. For any given L, as well as the maximum path length k in the network (i.e., maximum number of elements stored in a bloom filter), the optimal setting of H is $\lfloor \frac{L}{k} \ln 2 \rfloor$ such that minimizes the false negative probability [3]. We empirically investigate the setting of L in CSPR using the CitySee packet trace. We define a path group to be *conflicted* if not all packets in this group travel the same path, and use $\textit{conflict ratio} = \frac{\#\ of\ conflicted\ groups}{\#\ of\ all\ groups} \times 100\%$ to measure the misclassification rate. As the maximum path length (excluding the source and destination) observed in the trace is 10, the optimal H should be $\lfloor \frac{L}{10} \ln 2 \rfloor$. Fig. 7 depicts the conflict ratios when L varies from 16 to 56 with corresponding optimal H. From the figure, we see that the conflict ratio is generally not high, i.e., $< 9\%$. In particular, when L is larger than 40, the conflict ratio is smaller than 0.1%. With a balance between the classification accuracy and packet overhead, L in CSPR is set to 48 and H is thus 3 for the trace. An optimized setting of L and H for each individual node according to its maximum path length will further reduce the overall overhead, which will be explored in future.

In fact, as long as sufficient packets, truly belonging to the same path group, are received, our path reconstruction method (detailed in the following) can still recover the path even certain misclassified packets are mixed in the reconstruction. Moreover, after a routing path is recovered, a path verification component is used to further verify the reconstruction correctness, which can also eliminate all misclassified packets from the group as well. As a result, CSPR is not vulnerable to packet misclassification, while accurate classification is still preferred as high accuracy ensures that more paths could be recovered with shorter latency.

Path reconstruction. Based on the encoded measurements in received packets, CSPR recovers the path for a path group using compressive sensing technique. Concretely, for one path group, CSPR solves $Y = \Phi\mathbf{s}$ to obtain the path vector $\mathbf{s}$ for path recovery. Each element in Y, denoted as y_i, is the aMsr value of a re-

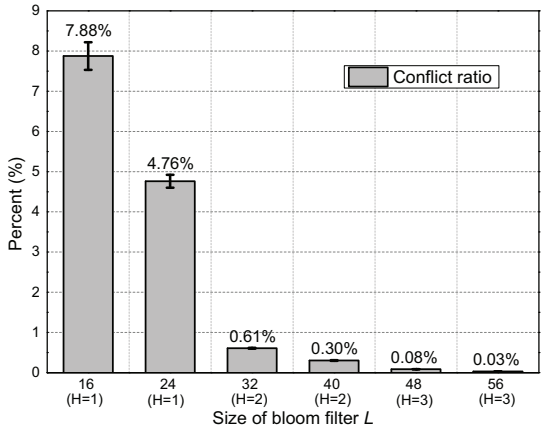


Figure 7: Conflict ratios under various combination setting of bloom filter size L and number of hash functions H for the CitySee packet trace.

ceived packet i. The corresponding row in Φ is represented by $\phi_i = [\alpha_{i,1}, \alpha_{i,2}, \cdots, \alpha_{i,N}]$, where $\alpha_{i,j}$ is the (SEQ_i *mod* $|\mathcal{D}_j|$)-th coefficient in the coefficient dictionary $\mathcal{D}_j$ of node j and SEQ_i is the sequence number of packet i. If the elements in $\mathbf{s}$ are known, the product of ϕ_i and $\mathbf{s}$ replays the updating process of the aMsr field along the path of packet i. Therefore, $y_i = \phi_i \mathbf{s}$. For the path reconstruction problem, both y_i and ϕ_i are known while the N elements in $\mathbf{s}$ are unknowns to be determined. In principle, N independent equations are needed to obtain $\mathbf{s}$, and each equation $y_i = \phi_i \mathbf{s}$ corresponds to one received packet.

Since path vector $\mathbf{s}$ is sparse, it can be recovered using M rather than N equations by leveraging compressive sensing. Therefore, as long as M packets are accumulated by one path group, the vector $\mathbf{s}$ can be recovered. We use the aMsr values from M packets to form an M-dimension column vector Y, i.e., $Y = [y_1, y_2, ..., y_M]^T$. For each packet i out of M packets, we further use its sequence number SEQ_i to select coefficients from coefficient dictionaries of all nodes in the path representation space $\mathbb{N}$ to construct $\phi_i = [\alpha_{i,1}, \alpha_{i,2}, \cdots, \alpha_{i,N}]$. All ϕ_i together form an $M \times N$ matrix:

$$\Phi = \begin{bmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_M \end{bmatrix} = \begin{bmatrix} \alpha_{1,1} & \alpha_{1,2} & \cdots & \alpha_{1,N} \\ \alpha_{2,1} & \alpha_{2,2} & \cdots & \alpha_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{M,1} & \alpha_{M,2} & \cdots & \alpha_{M,N} \end{bmatrix}. \quad (1)$$

Since all coefficients in Φ follow the Gaussian distribution, the measurement matrix Φ satisfies the RIP condition [20]. In addition, if the number of packets in the path group is sufficient, at least $ck\log(\frac{N}{k})$, we can apply the compressive sensing technique to recover the path vector $\mathbf{s}$. A variety of compressive sensing solvers can be used to obtain $\mathbf{s}$. However, most of them provide no confidence for the reconstruction quality. As a result, $\mathbf{s}$ might not be correctly reconstructed. In CSPR, since the sparsity of $\mathbf{s}$, i.e., path length indicated by the pLen field, is known prior to the path reconstruction, we adopt a more advanced solver CoSaMP [24] that requires the vector sparsity as input. If CoSaMP outputs a recovered result, the result is correct with a high probability. Otherwise, CoSaMP outputs "Fail" instead. Furthermore, CoSaMP can tolerate certain noises mixed in Y, and the path vector can be recovered even some misclassified packets are included. In particular, CoSaMP could recover $\mathbf{s}$ if the number of correctly classified packets is greater than $M_l = ck\log(\frac{N}{k})$ when $c = 1.5$ according to recent study [5].

In CSPR, after a packet is received, if the path group this packet belonging to has not recovered its path yet, the server checks whether the number of packets in the group exceeds M_l. The path reconstruction will be performed if the threshold is reached. If CoSaMP returns "Fail", more packets are expected. If CoSaMP returns a valid result, CSPR executes path verification component to further ensure its correctness.

Path verification. Given a recovered path and a packet, the path verification component verify whether the recovered path is valid for the packet via path vector $\mathbf{s}$ of the recovered path and aMsr value of the packet. More precisely, for a packet with sequence number SEQ, we calculate the product of ϕ and $\mathbf{s}$, where $\phi = [\alpha_1, \alpha_2, \cdots, \alpha_N]$ and α_j in ϕ is the (SEQ *mod* $|\mathcal{D}_j|$)-th coefficient in $\mathcal{D}_j$, $j = 1, 2, \cdots, N$. As coefficients are randomly gaussian, it is highly impossible for two packets traveling two different paths yet leading to the same aMsr value. Therefore, if $\phi \cdot \mathbf{s}$ equals to the aMsr value of the packet, the recovered path is valid for this packet.

When a path p is newly reconstructed, this component is executed to verify the correctness of path p for a path group. It is also used to eliminate all misclassified packets due to bloom filter collision. If path p is only valid for the minority of packets in the group (e.g., $\leq 20\%$), the path recovery is considered to be failed, e.g., an incorrect output from CoSaMP. The path reconstruction will be performed after more packets are received. Otherwise, path p is viewed as the correct path, and the corresponding *Flag* of this group is changed to be *true*. However, we exclude all packets failed in the aMsr check from current group and form a new group, with *Flag* as *false*, for those packets. Since the new group has the same 3-tuple key with current group, we introduce gIdx as the secondary key, an auto-increment key, to further distinguish entries in database with the same 3-tuple key. For example, if any 3-tuple key corresponds to one group g_1 only, the gIdx key of this group is 0. Later, if new groups g_2 and g_3 are formed successively, their gIdx keys would be 1 and 2 respectively, both of which are automatically increased by one based on the gIdx key of previous group.

The recovered paths will benefit all future packets traveling on them. When a group with recovered path receives a packet, CSPR just simply invokes the path verification component to check whether the packet truly traveled the recovered path. If yes, this packet obtains its path immediately. In CSPR, for a given 3-tuple key, path groups are matched with the packet following the ascending order of gIdx keys. At worst, the packet might be assigned to a newly formed path group. Thanks to identifying each individual path, a large number of packets could have their path with no recovery delay and meanwhile CSPR avoids huge computation overhead, which will be demonstrated in our evaluations.

3.4 Optimization

In this subsection, we propose two optimization techniques to improve the performance of our basic design.

Reduction of path representation space. This optimization aims to reduce the number of elements in a representation space by continuously monitoring the network topology. The minimum number of packets required to recover a length of k path is $ck\log(\frac{N}{k})$, which is proportional to the space size N. The basic design utilizes all N nodes in the network to form the space. This component tries to reduce the space size for each path group and thus reduces its needed packets for path reconstruction. For any node i, CSPR maintains a first-hop receiver set $\mathbb{S}_i$, which is learnt from received packets (via the *first-hop receiver* field in packet header) and recovered paths (the next hop of node i along a path). All elements in

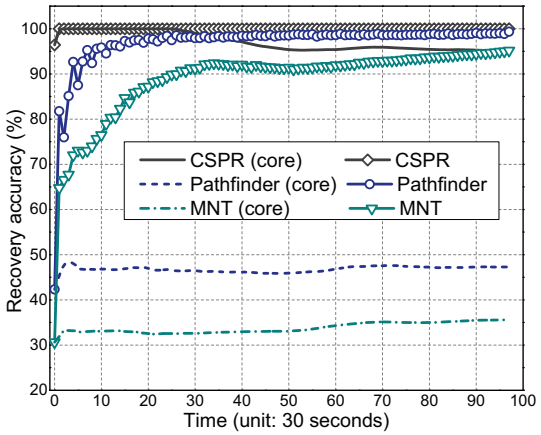


Figure 8: The packet path reconstruction progresses with the core and complete methods of each approach.

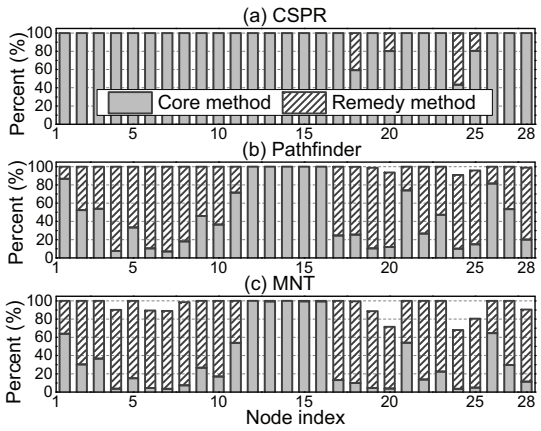


Figure 9: The portion of packet paths recovered by core or remedy methods.

$\mathbb{S}_i$ have ever received packets from node i. The reduced representation spaces for all path groups with the sArr attribute as node i are the same, denoted as $\overline{\mathbb{N}}_i$. Elements of $\overline{\mathbb{N}}_i$ are added in an iterative manner: 1) elements in $\mathbb{S}_i$ belong to $\overline{\mathbb{N}}_i$; 2) elements in $\mathbb{S}_j$, where $j \in \overline{\mathbb{N}}_i$, belong to $\overline{\mathbb{N}}_i$.

We take node 9 in Fig. 4 as an example to illustrate the formation of $\overline{\mathbb{N}}_9$. The first-hop receiver set $\mathbb{S}_9$ of node 9 contains 5 and 8. By iteratively including all first-hop receivers of nodes in $\overline{\mathbb{N}}_9$, $\overline{\mathbb{N}}_9$ finally contains 2, 5, 8 and 10. Compared with original space $\mathbb{N}$, the size of $\overline{\mathbb{N}}_9$ is reduced from 10 to 4. From our investigation on the CitySee packet trace, we find the average and maximum reduced representation space sizes are only 39 and 104 respectively, which are much smaller than the total number of nodes 245 in the network.

Even with sufficient packets, path reconstruction based on the reduced representation space might be unsuccessful unless all intermediate nodes of the path are included in the reduced space. As it is hard for CSPR to explicitly determine whether all intermediate nodes are included, we thus use the reduced representation space as a backup scheme. For each path group, CSPR always use the original N-dimension representation space for the reconstruction. If the reconstruction fails, CSPR recovers one more time using the reduced space. If the second reconstruction succeeds, CSPR will launch the path verification component to verify the correctness of the recovered path; Otherwise, CSPR waits for more packets and performs the next round reconstruction, still starting from the N-dimension representation space.

With more reconstructed routing paths, we could infer the common intermediate nodes between the unrecovered paths and the paths that have been reconstructed, which provides another opportunity to improve our design, i.e., reducing the path vector sparsity such that the remaining paths can be recovered by fewer packets. Such an optimization chance will be explored in our future work.

Heuristic path scanning. For those path groups with insufficient accumulated packets even after a long time, this component is designed to scan possible paths for them based on the learnt network topology. It is triggered when the path reconstruction deadline of a path group is approaching. As this scheme is relatively computation intensive, it recovers path not only for the group which triggers its execution, but also for other unrecovered groups whose paths have the same source, sArr, as this group at the same time.

Starting from the source node i of the path group that triggers heuristic path scanning, CSPR builds a directed graph covering all nodes in the reduced representation space $\overline{\mathbb{N}}_i$. For any two nodes a and b in $\overline{\mathbb{N}}_i$, if node b is in the first-hop receiver set of node a, i.e., $b \in \mathbb{S}_a$, CSPR adds an arrow from a to b. This component enumerates all possible paths from source node i to the sink in the directed graph. For each enumerated path, CSPR treats it as a newly reconstructed path and applies the path verification procedure to check whether it is a valid path for some unrecovered group whose path is originated from node i. As a result, heuristic path scanning can recover paths for multiple path groups.

4. PERFORMANCE EVALUATION

In this section, we evaluate the performance of CSPR with comparisons to two state-of-the-art approaches based on a 29 TelosB mote testbed and the CitySee packet trace.

4.1 Evaluation setup

Compared approaches. We compare CSPR to the two state-of-the-art approaches, Pathfinder [14] and MNT [16]. For fairness, we implement the path speculation in Pathfinder [14] as a remedy method for both Pathfinder and MNT to improve their performances when anchors are lost. We add 1-byte XOR field into packet header for MNT as checksum, similar with Pathfinder, to verify the reconstructed paths. For all the three approaches, we define the same path recovery delay bound δ to meet practical requirements. In general, the three approaches will reconstruct packet paths following their core principles (referred as *core method*), and employ the remedy methods (heuristic path scanning for CSPR and path speculation for Pathfinder and MNT, referred as *remedy method*) to reconstruct those failed packets when path recovery delay bound is approaching. In the following experiments and simulations, we set the bound $\delta = 50$, which means a remedy method will be employed to search possible routing paths when another 50 packets of the same source are received. Small δ will reduce the overall recovery delay but trigger much more computation overhead. All approaches are running on a desktop PC with dual-core 3.16GHz CPU and 4GB RAM. The execution of remedy methods have a time limit of 1 second to avoid excessive computation overhead.

Performance metrics. The *recovery accuracy* for each node is calculated as $\frac{\#\,of\ correct\ recovered\ pkts}{total\ \#\,of\ pkts\ received} \times 100\%$. Similarly, the recovery *false positive* is computed as $\frac{\#\,of\ false\ positive\ recovered\ pkts}{total\ \#\,of\ pkts\ received} \times 100\%$ for each node. Intuitively, the approach with higher recovery accuracy yet lower false positives is expected for per-packet path reconstruction in WSNs.

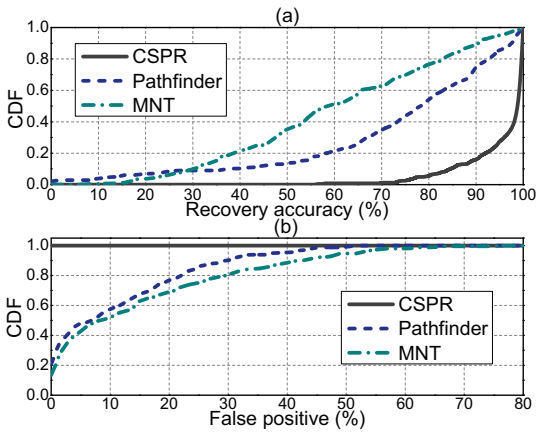


Figure 10: (a) CDF of path recovery accuracy. (b) CDF of false positive for each approach.

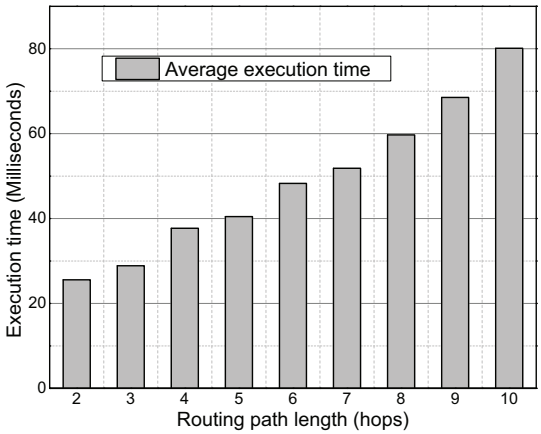


Figure 11: The program execution time of path reconstruction by CSPR versus the routing path length.

4.2 Testbed experiments

We implement CSPR on TelosB mote and use a 29-node testbed to validate its feasibility and applicability. 29 TelosB motes are uniformly distributed in a square area. One node acts as the sink and is placed at the top-left corner. Due to the limitation of experimental space, we configure the transmission power of each TelosB mote to the minimum level and thus the communication range of each sensor node would be about 15 centimeters. CSPR is implemented at the application layer on TinyOS 2.1.2 with CTP [15] as the data collection protocol. Each node generates packets randomly with an average inter packet interval one second. Within the network, each intermediate node along a path updates the packet header following the requirements of each approach. The sink receives packets from network and separately executes the three approaches to reconstruct packet paths. The actual path is recorded in packets as ground truths. The experiment lasts for about 50 minutes and collects 60000 packets.

The sink records the path reconstruction progress of each approach every 30 seconds. Concretely, for each approach, the portions of packets accurately recovered with only the core method and the complete approach are both computed. The evolution of path reconstruction progress of each approach is plotted in Fig. 8. From the figure, we find that Pathfinder and MNT can only reconstruct 47% and 36% packet paths based on their core methods, respectively. Most reconstruction failures are caused by packet losses (about 38% packet loss rate during experiment). With no fixed packet generation interval, Pathfinder loses its accuracy in locating anchors which further harms the performance. Assisted by the remedy method (i.e., path speculation), they can achieve similar accuracy as the core method of CSPR, i.e., $> 95\%$. With heuristics path scanning, CSPR achieves accuracy of 100%. Fig. 9 further presents the detailed per-node path reconstruction performance. 24 nodes can recover their packet paths just by the core method of CSPR. With respect to the other two approaches, however, most nodes achieve a good recovery accuracy through the remedy method.

4.3 Trace-driven simulations

To evaluate the scalability and efficiency of CSPR in practical large-scale and more dynamic networks, we conduct extensive trace-driven simulations by leveraging the practical packet trace from the real-deployed and large-scale WSN CitySee [22], as introduced in Section 2.2. The packet trace includes packets from a network of 245 nodes with 174829 packets in total.

We apply the three approaches, i.e., CSPR, Pathfinder and MNT, to reconstruct per-packet path in the trace, and summarize the CDFs of path recovery accuracy and false positive, for each approach, in Fig. 10. From Fig. 10 (a), we find that CSPR outperforms Pathfinder and MNT with significant advantages. Pathfinder and MNT perform poorly in the practical packet trace, which experiences severe topology dynamics and packet losses as revealed in Section 2.2, with recovery accuracies of most nodes falling in the range between 50% and 80%. Only 23% and 45% nodes for MNT and Pathfinder, respectively, can achieve an accuracy $\geq 80\%$, while CSPR makes 95% nodes reach such level. Furthermore, about 85% nodes achieve a high accuracy $\geq 90\%$. Overall, the average recovery accuracies for MNT, Pathfinder and CSPR are 62%, 74%, 96%, respectively. In Fig. 10 (b), we see non-negligible false positives in Pathfinder and MNT, both of which verify the recovered paths mainly based on the XOR field in packets. Pathfinder checks the result with more information recorded in packet, and thus has fewer false positives than MNT. Such path verification manners result in on average 11% and 15% false positive for MNT and Pathfinder, respectively. Thanks to the path verification component enabled by the aMsr field, CSPR has no false positive. The correctness of recovered routing paths is very important as they might be applied to other applications to provisioning guaranteed QoS [34], such as network diagnosis [18] [26] and per-hop per-packet delay [13].

CSPR is a lightweight path reconstruction approach, and the overhead of the compressive sensing recovery, e.g., CoSaMP, is negligible on a server. The time complexity of CoSaMP is $O(\lambda M_l N)$ [24], where M_l is the number of measurements needed, N is the network size, and λ is the maximum iterations of CoSaMP, which is set as 100 in our implementation. In Fig. 11, we measure the program execution time as the performance metric to further understand the computation overhead of our design. For different routing path lengths, we plot the reconstruction delays, which are mainly caused by the compressive sensing recovery. Fig. 11 shows that the overhead due to compressive sensing recovery is negligible which is less than 100 milliseconds for all routing path lengths observed in the experiments.

Because of the packet classification mechanism in CSPR, a great portion of packets can recover their paths at a negligible cost of only executing path verification with the recovered paths in path groups. In Fig. 12, we plot the portion of packets that benefit from such mechanism. After an accumulation phase of recovered paths, CSPR can find correct paths for about 65% packets. As a result,

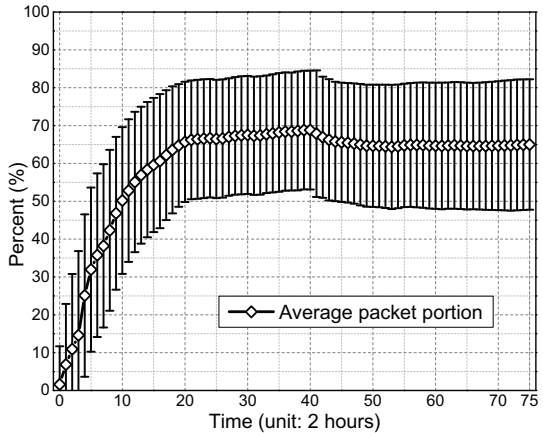


Figure 12: The portion of packets benefited from already recovered routing paths.

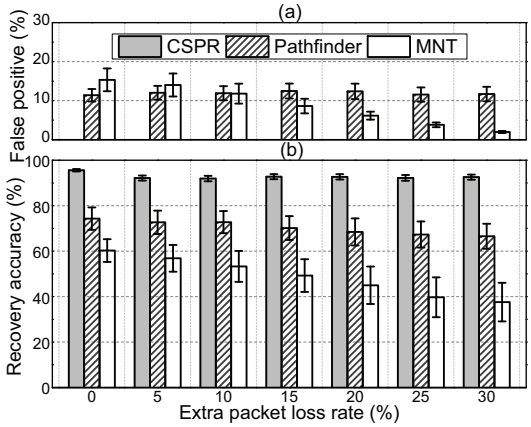


Figure 13: The recovery accuracies and false positives of the three approaches with various extra packet loss rates.

CSPR avoids repeating reconstruct those frequent traveled paths, and thus saves abundant of computation.

In Section 2.2, we have already observed about 20% natural packet losses. To examine the impacts of severer packet losses to the path reconstruction performance of each approach, we fabricate extra packet losses by randomly removing packets from the original trace. We present the recovery accuracy and false positive of each approach with various extra packet loss rates in Fig. 13. Note that each value in Fig. 13 is an average of 10 runs. The extra packet loss rate labeling *"0"* corresponds to the original trace. With more packets lost, the performance of MNT drops quickly. MNT becomes to be not applicable, with accuracy $< 50\%$, when the extra packet loss rate beyond 15%. The performance of Pathfinder is also affected, with accuracy dropping from 74% to 66%. These results illustrate that packet losses indeed have a strong impact on the performances of approaches relying on inter-packet correlation. When anchors lost, Pathfinder and MNT use the remedy method to search possible paths for packets and thus introduce false positives, due to their relaxed path verification manners. From Fig. 13 (a), we observe non-negligible false positives for Pathfinder and MNT with averages ranging from 4% to 15%. On the contrast, CSPR is insensitive to packet losses and can achieve stable accuracy $> 92\%$ with no false positive.

Packet overhead. In addition to some default information in the packet header, e.g., source address, sequence number, hop count and first-hop receiver, each path reconstruction approach further adds some necessary contents to each packet for the path recovery. Assuming that a node ID is of size 2 bytes. Then the packet overhead of MNT is 5 bytes including the 1-byte XOR and 4-byte timestamp. The packet overhead of Pathfinder is about 7 bytes including the 1-byte XOR, 2 IDs of changed parents and a bit vector as the same length of packet path. The packet overhead of CSPR could be variable in different sizes of networks. CSPR includes a measurement of 2 bytes (i.e., aMsr) and an L-bit bloom filter (i.e., bFlt). Note that L is related with the maximum path length in the network and could be small, e.g., $L = 48$ (6 bytes) for the CitySee packet trace with the maximum path length 12 hops, which actually corresponds to a large-scale network.

5. RELATED WORK

We review the related works on routing path reconstruction, network tomography and network diagnosis, mainly conducted in WSNs, in this section.

Routing path reconstruction. CAPTRA [27] identifies a packet path through the coordinations among nodes in a network-wide scale. Alam *et al.* [2] adopt a probabilistic packet marking technique to trace the provenance of a packet. Both of the works can not realize per-packet path reconstruction as CSPR. PathZip [19] compresses path information of a packet into a hash value, and obtains the packet path by matching all possible paths with the hash value. As the computation complexity grows exponentially with the network size, PathZip may not scale to work with large networks. Pathfinder [14] and MNT [16], the two state-of-the-art approaches, reconstruct packet path relying on inter-packet correlation. Their performances, however, are severely influenced by topology dynamics and packet losses, just as demonstrated in Section 2.2. Different from them, CSPR is insensitive to network dynamics and lossy links due to its distinct design.

Network tomography. Network tomography in wired networks has been well studied and tremendous approaches have been proposed to investigate the internal behaviors [8]. By actively generating probe packets from network nodes, network tomography mainly aims to recover the network topology or infer some link-level characteristics [21], e.g., delay or packet loss. Restricted by the available resources, extensive probes are prohibited for network topography in WSNs. Recently many works have been proposed to achieve network tomography in WSNs by leveraging statistical methods [25], group testing [7], compressive sensing [31] and optimization methods [13]. Compared to those works, CSPR reconstructs routing path for each individual packet without triggering extra probe packets.

Network diagnosis. Network diagnosis aims at inferring the root cause for abnormal networking symptoms and maintaining the health of deployed WSNs. Relying on the collected system metrics from network, Sympathy [26] pinpoints the root cause of network failures through a decision tree. PAD [18] captures abnormal events and infers the root cause for the observed abnormity in a probabilistic manner. The active packet marking scheme in PAD can only recover one routing path for each source node, while the underlying network topology is required to be relatively stable. PD2 [6] is a data-centric approach that locates performance problems based on data flows. D2 [10] detects and diagnoses anomaly by mining network symptoms. AD [23] exploits the correlation among different system metrics to discover silent failures. Existing works in this category are orthogonal to CSPR, and are potentially benefitted from the outputs of CSPR for more accurate and fine-grained diagnostic results.

6. CONCLUSION

In this paper, we present the CSPR, a compressive sensing based path reconstruction approach. Different from the state-of-the-art approaches, CSPR is inherently insensitive to network dynamics and lossy links. Extensive evaluations through both testbed-based experiments and trace-driven simulations show that CSPR outperforms the state-of-the-art approaches in various network settings.

7. ACKNOWLEDGEMENTS

We would like to thank the anonymous reviewers for valuable and insightful comments. This work is supported in part by National High Technology Research and Development Program of China (863) under Grant No.2012AA101701; National Key Research & Development Program under Grant No.2013BAK01B04; Project of Innovative Team of Digital Cultural Media Technology of Zhejiang Province under Grant No.2010R50040; NSFC No.61303233. This work is also partially supported by Singapore MOE AcRF Tier 2 Grant MOE2012-T2-1-070, and NTU Nanyang Assistant Professorship (NAP) Grant M4080738.020. We also acknowledge the support of practical packet trace from the CitySee project [22].

8. REFERENCES

[1] TelosB mote datasheet. `http://www.willow.co.uk/TelosB_Datasheet.pdf`.

[2] S. Alam and S. Fahmy. A practical approach for provenance transmission in wireless sensor networks. *Ad Hoc Networks*, 2013.

[3] A. Broder and M. Mitzenmacher. Network applications of bloom filters: a survey. *Internet Mathematics*, 1(4):485–509, 2004.

[4] E. J. Candès, J. Romberg, and T. Tao. Robust uncertainty principles: exact signal reconstruction from highly incomplete frequency information. *IEEE Transactions on Information Theory*, 52(2):489–509, 2006.

[5] E. J. Candès and M. B. Wakin. An introduction to compressive sampling. *IEEE Signal Processing Magazine*, 25(2):21–30, 2008.

[6] Z. Chen and K. G. Shin. Post-deployment performance debugging in wireless sensor networks. In *IEEE RTSS*, 2009.

[7] M. Cheraghchi, A. Karbasi, S. Mohajer, and V. Saligrama. Graph-constrained group testing. *IEEE Transactions on Information Theory*, 58(1):248–262, 2012.

[8] A. Coates, A. O. Hero III, R. Nowak, and B. Yu. Internet tomography. *IEEE Signal Processing Magazine*, 19(3):47–65, 2002.

[9] D. Dong, M. Li, Y. Liu, X.-Y. Li, and X. Liao. Topological detection on wormholes in wireless ad hoc and sensor networks. *IEEE/ACM Transactions on Networking*, 19(6):1787–1796, 2011.

[10] W. Dong, C. Chen, J. Bu, X. Liu, and Y. Liu. D2: anomaly detection and diagnosis in networked embedded systems by program profiling and symptom mining. In *IEEE RTSS*, 2013.

[11] W. Dong, Y. Liu, Y. He, and T. Zhu. Measurement and analysis on the packet delivery performance in a large scale sensor network. In *IEEE INFOCOM*, 2013.

[12] D. L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006.

[13] Y. Gao, W. Dong, C. Chen, J. Bu, T. Chen, M. Xia, X. Liu, and X. Xu. Domo: passive per-packet delay tomography in wireless ad-hoc networks. In *IEEE ICDCS*, 2014.

[14] Y. Gao, W. Dong, C. Chen, J. Bu, G. Guan, X. Zhang, and X. Liu. Pathfinder: robust path reconstruction in large scale sensor networks with lossy links. In *IEEE ICNP*, 2013.

[15] O. Gnawali, R. Fonseca, K. Jamieson, D. Moss, and P. Levis. Collection tree protocol. In *ACM SenSys*, 2009.

[16] M. Keller, J. Beutel, and L. Thiele. How was your journey?: uncovering routing dynamics in deployed sensor networks with multi-hop network tomography. In *ACM SenSys*, 2012.

[17] X.-Y. Li, Y. Wang, H. Chen, X. Chu, Y. Wu, and Y. Qi. Reliable and energy-efficient routing for static wireless ad hoc networks with unreliable links. *IEEE Transactions on Parallel and Distributed Systems*, 20(10):1408–1421, 2009.

[18] Y. Liu, K. Liu, and M. Li. Passive diagnosis for wireless sensor networks. *IEEE/ACM Transactions on Networking*, 18(4):1132–1144, 2010.

[19] X. Lu, D. Dong, X. Liao, and S. Li. PathZip: packet path tracing in wireless sensor networks. In *IEEE MASS*, 2012.

[20] C. Luo, F. Wu, J. Sun, and C. W. Chen. Compressive data gathering for large-scale wireless sensor networks. In *ACM MobiCom*, 2009.

[21] L. Ma, T. He, K. K. Leung, D. Towsley, and A. Swami. Efficient identification of additive link metrics via network tomography. In *IEEE ICDCS*, 2013.

[22] X. Mao, X. Miao, Y. He, X. Li, and Y. Liu. CitySee: urban CO_2 monitoring with sensors. In *IEEE INFOCOM*, 2012.

[23] X. Miao, K. Liu, Y. He, Y. Liu, and D. Papadias. Agnostic diagnosis: discovering silent failures in wireless sensor networks. In *IEEE INFOCOM*, 2011.

[24] D. Needell and J. A. Tropp. CoSaMP: iterative signal recovery from incomplete and inaccurate samples. *Applied and Computational Harmonic Analysis*, 26(3):301–321, 2009.

[25] H. X. Nguyen and P. Thiran. Using end-to-end data to infer lossy links in sensor networks. In *IEEE INFOCOM*, 2006.

[26] N. Ramanathan, K. Chang, R. Kapur, L. Girod, E. Kohler, and D. Estrin. Sympathy for the sensor network debugger. In *ACM SenSys*, 2005.

[27] D. Sy and L. Bao. CAPTRA: coordinated packet traceback. In *ACM/IEEE IPSN*, 2006.

[28] S. Tang and J. Wu. Qute: quality-of-monitoring aware sensing and routing strategy in wireless sensor networks. In *ACM MobiHoc*, 2013.

[29] J. Wang, W. Dong, Z. Cao, and Y. Liu. On the delay performance analysis in a large-scale wireless sensor network. In *IEEE RTSS*, 2012.

[30] J. Wang, Y. Liu, Z. Li, W. Dong, and Y. He. QoF: towards comprehensive path quality measurement in wireless sensor networks. In *IEEE INFOCOM*, 2011.

[31] W. Xu, E. Mallada, and A. Tang. Compressive sensing over graphs. In *IEEE NFOCOM*, 2011.

[32] Y. Yang, Y. Xu, X. Li, and C. Chen. A loss inference algorithm for wireless sensor networks to improve data reliability of digital ecosystems. *IEEE Transactions on Industrial Electronics*, 58(6):2126–2137, 2011.

[33] T. Zhu, W. Dong, Y. He, Q. Ma, L. Mo, and Y. Liu. Understanding routing dynamics in a large-scale wireless sensor network. In *IEEE MASS*, 2013.

[34] Y. Zhu and L. M. Ni. Probabilistic approach to provisioning guaranteed QoS for distributed event detection. In *IEEE INFOCOM*, 2008.